

Lewisham Household Arrears Risk Model — Approach and Evaluation

Submission note

This document covers Parts 1, 2, and 4 of the exercise. Part 3 — production-grade code for one stage of the pipeline — is the training stage, submitted as the contents of the linked repository (primarily [src/arrears_risk_model/train.py](#) and its dependencies: [config.py](#), [data.py](#), [features.py](#), [models.py](#), [evaluate.py](#)).

The work builds on a prior analytical project that established the feature choices, modelling approach, and model usage as a Jupyter notebook. I've used this as a basis to re-write the code to production standards as much as was possible.

The submission entails configuration, schema validation, ingestion, feature engineering, training with grid search, evaluation, prediction CLI, model card, tests, and CI — and adds model checks.

- Repository: <https://github.com/nadinespy/Arrears-Risk-Model>
- Notebooks (in notebooks/):
 - 01_eda_and_imd.ipynb,
 - 02_clustering.ipynb,
 - 03_modelling.ipynb
- Model card: [docs/model_card.md](#)

Part 1 — Problem Outline

Situation

Lewisham Borough Council's housing and benefits team manages a caseload of approximately 17,831 households receiving housing benefit or Universal Credit support. Rent arrears create significant hardship for tenants — and downstream costs for the council through legal recovery, eviction proceedings, and homelessness support — but tend to be addressed reactively, after they have already escalated.

Proactive outreach at early stages of financial difficulty is more effective and less costly, but the caseload volume makes blanket outreach impractical. The team needs a principled way to prioritise which households to contact first.

The use case is batch (weekly or monthly priority list), not real-time scoring — which shapes downstream design: no API latency requirements, simpler serving, but stronger requirements on auditability and explanation quality, because every prediction will be reviewed by a human.

What the output is used for

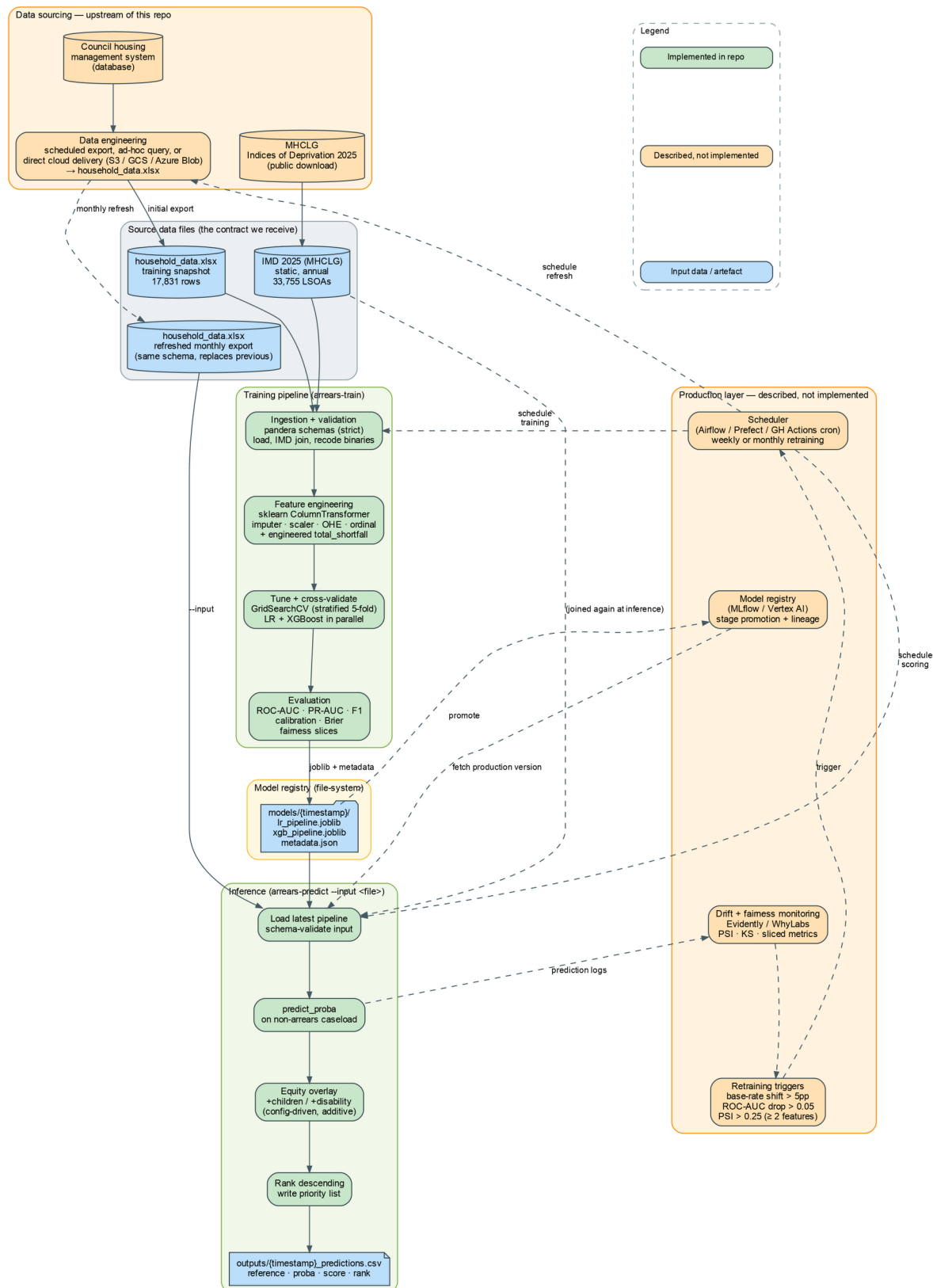
The model produces a **ranked priority list** of households by estimated arrears risk, run weekly or monthly. The list is used by caseworkers to triage proactive contact — phone calls, letters, referrals to debt advice services — before arrears develop or worsen.

The score is an input to caseworker judgement, not an automated decision. No household is contacted, referred, or denied a service solely on the basis of a model score.

Key user requirements

Requirement	Design response
Human review before any action	Batch output (CSV); no automated contact or decision pathway
Explain why a household is ranked highly	SHAP values computed per run; LR baseline for linear explainability
Equity — children and disability should receive a systematic boost	Configurable additive equity overlay applied before ranking
Fairness monitoring by protected characteristic	Descriptive sliced metrics (TPR, FPR, selection rate) by disability, household type, age band
Staff can adjust weights without touching code	All parameters in config/default.yaml; overridable via ARM_* env vars
Reproducible, auditable runs	Each training run writes metadata.json with metrics, config snapshot, library versions
Deployable by a small technical team	Single uv sync install; arrears-train and arrears-predict CLI entry points

Part 2 — Solution Architecture (high-level pipeline)



Stage-by-stage description

Stage 1 — Data ingestion and validation

Item	Detail
Sources	household_data.xlsx (council benefits/housing system, 17,831 rows, 21 columns); File_1_1oD2025_Index_of_Multiple_Deprivation.xlsx (MHCLG, OGL) joined on LSOA code
Libraries	pandas, openpyxl, geopandas (geographic analysis in notebooks)
Validation	pandera DataFrameSchema with strict=True on both raw inputs and the joined output — rejects unknown or missing columns at ingest time
Decisions	Strict schema validation chosen over silent coercion so that upstream data model changes (column renames, dtype changes) fail loudly rather than producing wrong results downstream
Assumptions	Data is exported as .xlsx from the council's housing management system; LSOA codes are consistent between the household extract and the IMD file

Stage 2 — Data preparation

Item	Detail
Operations	Many-to-one join on lsoa21cd; Yes/No binary columns recoded to 0/1; arrears flag/amount consistency check (logged, not filtered)
Libraries	pandas, pandera
Decisions	Recoding and join are pure transformations with no model-fitting — done outside the sklearn pipeline so they can be inspected and tested independently. Imputation (one missing value in ben_cap_amount) is deferred to the sklearn pipeline to avoid leakage

Stage 3 — Feature engineering

Item	Detail
Engineered features	total_shortfall = lha_shortfall_weekly + ben_cap_amount (computed inside the pipeline via FunctionTransformer)
Continuous	Zero imputation for missing ben_cap_amount (matches the total_shortfall engineering, which treats a missing cap as no deduction); StandardScaler for LR only
Categorical	OneHotEncoder(handle_unknown="ignore") for LR; OrdinalEncoder(handle_unknown="use_encoded_value") for XGBoost
Ordinal	OrdinalEncoder with explicit category ordering; StandardScaler for LR only

Item	Detail
Libraries	scikit-learn (ColumnTransformer, Pipeline, FunctionTransformer, SimpleImputer)
Decisions	All feature engineering lives inside a single sklearn Pipeline object so the full preprocessing + model can be serialised as one joblib artefact, eliminating train/serve skew. XGBoost does not receive scaling (trees are scale-invariant); LR does. Separate preprocessors are used per model but share the same column definitions from config

Stage 4 — Model training and evaluation

Item	Detail
Models	Logistic regression (interpretable baseline) and XGBoost (primary scoring model) trained in parallel
Imbalance handling	LR: class_weight="balanced"; XGBoost: scale_pos_weight recomputed per fit by XGBClassifierAutoSPW so every CV fold derives its own ratio (~3:1)
Hyperparameter tuning	GridSearchCV with stratified inner CV on the training set; refit by ROC-AUC. Grids are config-driven (hyperparameter_search.lr_grid, xgb_grid); enable/disable via hyperparameter_search.enabled. The held-out test set never participates in tuning
Validation	Stratified 80/20 train/test split; held-out test set withheld before any fitting or tuning. Cross-validation on the training set provides mean $\pm$ std across folds; tuning, when enabled, replaces this with the best configuration's CV statistics
Metrics	ROC-AUC, PR-AUC, F1 (threshold 0.5), Brier score, calibration curve
Fairness	Descriptive sliced metrics (selection rate, TPR, FPR, precision, ROC-AUC) computed per value of disability, household_type, age_bracket — surfaced in metadata.json, no automated gate
Libraries	scikit-learn, xgboost, shap, pydantic (typed result objects)
Serialisation	joblib — one .joblib per model per run, stored in models/{timestamp}/ alongside metadata.json
Decisions	Two models rather than one: LR provides linear explainability and a calibrated probability baseline; XGBoost provides better ranking performance. Keeping both lets the council choose the operating model per use case. No automated fairness deployment gate — this is a deliberate policy decision (see Chouldechova 2017: satisfying all fairness criteria simultaneously is mathematically impossible when base rates differ across groups)

Stage 5 — Scoring and output generation

Item	Detail
Process	Load latest pipeline; score all households with predict_proba; apply equity overlay; sort descending by composite score; write ranked CSV
Equity overlay	Additive: +config.equity.children if household has dependent children or FSM-eligible; +config.equity.disability if disabled member. Default +0.05 each — configurable in YAML without code changes
Output	outputs/{timestamp}_predictions.csv with columns: reference, predicted_proba, composite_score, rank
Libraries	pandas, joblib
Decisions	Additive overlay chosen over multiplicative because it is interpretable and auditable: the uplift is a fixed amount visible in the score column, not a hidden transformation. Additive form is only meaningful if the model is reasonably calibrated — Brier score is tracked to confirm this

Stage 6 — Infrastructure and deployment

Item	Detail
Deployment mode	Batch-only; no real-time API serving (not needed for weekly/monthly prioritisation)
Entry points	arrears-train and arrears-predict — installed scripts via pyproject.toml; arrears-predict accepts --input <path> to score a fresh monthly export without editing config or env vars points (equivalent to ARM_PATHS__HOUSEHOLD_DATA=<path>).
Configuration	config/default.yaml + ARM_* environment variable overrides (double-underscore nested delimiter); validated by pydantic-settings
Dependency management	uv + pyproject.toml; reproducible with uv sync
CI	GitHub Actions: ruff lint + pytest on push and PR to main
Testing	pytest with synthetic 50-row fixtures; no real data required in CI
Infrastructure assumptions	A Linux server or cloud VM with Python 3.12 and access to the council's data exports is sufficient. No GPU, no container orchestration, no streaming infrastructure required for batch operation

Architecture decisions summary

- **Batch over real-time:** the use case is weekly/monthly prioritisation, not instant scoring. Batch eliminates latency, infrastructure complexity, and the need for an always-on service.
- **sklearn Pipelines:** all preprocessing + model in one serialisable object eliminates train/serve skew and makes the pipeline testable as a unit.
- **Two models:** LR for explainability and audit; XGBoost for performance. The council can switch between them via `--model lr|xgb`.
- **Config-driven equity overlay:** weights are YAML-configurable so the council can adjust policy without touching code, and every change is auditable through version control.
- **No fairness gate:** mathematically impossible to satisfy all criteria simultaneously — descriptive monitoring is more honest than picking one criterion and optimising against it.

What's implemented in the repository vs. described as full production

Stage	In this repository	Described as full production
Configuration	pydantic-settings + config/default.yaml; ARM_* env overrides	—
Schema validation	pandera DataFrameSchema (strict) on inputs and joined frame	—
Data ingestion	pandas Excel reader; LSOA-level join; binary recoding	For this exercise, the data consists of a single excel file, for production this would be replaced by a scheduled export using a database (e.g., in a data warehouse such as Google Cloud)
Feature engineering	sklearn ColumnTransformer + FunctionTransformer; engineered total_shortfall	—
Model training	arrears-train CLI; GridSearchCV with stratified 5-fold inner CV	Triggered by an orchestrator (Airflow / Prefect / GitHub Actions cron) on a weekly or monthly cadence
Model registry	File-system: models/{timestamp}/ with joblib pipelines + metadata.json	Tagged registry with promotion stages and lineage (MLflow / Vertex AI / SageMaker Model Registry)
Inference	arrears-predict CLI batch run; equity overlay + ranked CSV output	FastAPI wrapper if real-time scoring ever becomes a requirement (not for this use case)
Evaluation	ROC-AUC, PR-AUC, F1, calibration,	—

Drift + fairness monitoring	Brier, fairness slices saved per run —	Evidently / WhyLabs / SageMaker Model Monitor — PSI, KS, sliced metrics over time, dashboards, alerting
Tests	pytest over synthetic 50-row fixtures — (no real data required in CI)	
CI	GitHub Actions: ruff + pytest on push — and PR	

The split is deliberate. Anything whose value is in demonstrated engineering craft — validation, encoding, modelling, evaluation, prediction, tests, CI — is in this repository. Anything that needs deployed infrastructure to be meaningful — orchestration, a tagged production registry, live drift dashboards, alerting — is described here and depicted with dashed edges in the diagram, not built. The boundary is shown explicitly so one can see exactly what would still need to be set up before this system runs unattended.

Part 4 — Model Evaluation

During development

Metrics

Metric	Rationale
ROC-AUC	Measures overall discrimination — the model's ability to rank arrears households above non-arrears households, regardless of threshold. The primary operating question is ranking quality, not classification at a fixed threshold
PR-AUC	Precision-recall area under curve; more informative than ROC-AUC at the operating point that matters in imbalanced settings (~25% positive rate). A high PR-AUC means the model can identify true positives with acceptable precision
F1 (threshold 0.5)	Harmonic mean of precision and recall at a standard threshold; included for comparability, though the operating threshold should be reviewed against capacity constraints rather than fixed at 0.5
Brier score	Measures probability calibration — how close predicted probabilities are to observed frequencies. Critical for the equity overlay, which assumes the raw probability is meaningful as a magnitude, not just a ranking signal

Validation strategy

- **80/20 stratified train/test split** — held-out test set withheld before any fitting or hyperparameter tuning
- **Hyperparameter tuning via GridSearchCV** — stratified inner CV on the training set, multi-metric scoring (ROC-AUC, PR-AUC, F1) with refit on a configured primary metric. Per-model grids live in config/default.yaml; tuning can be disabled in config (e.g. when running quick smoke checks)
- **5-fold stratified cross-validation** on the training set — when tuning is enabled, the reported CV mean $\pm$ std describes the best configuration; otherwise it describes the un-tuned default. Either way, every fold is a clean train/validate split with preprocessing fit per fold
- **Calibration curve (reliability diagram)** — visually confirms whether predicted probabilities are well-calibrated across the probability range

Fairness evaluation

Descriptive sliced metrics computed for each unique value of disability, household_type, and age_bracket:

- Selection rate (predicted positive rate at threshold)
- True positive rate (recall within slice)
- False positive rate within slice
- Precision within slice
- ROC-AUC within slice (where both classes present)

These are surfaced in metadata.json per run. There is no automated pass/fail gate — the choice of which fairness criterion to prioritise is a policy decision requiring input from council leadership, legal, and community stakeholders, not a modelling decision.

In live / production environments

Input monitoring

Check	Method	Threshold
Feature distribution shift	Population Stability Index (PSI) on all continuous and categorical input features; compare current scoring run against training distribution	PSI > 0.2 → flag for review; PSI > 0.25 → block run pending investigation
Score distribution shift	Kolmogorov-Smirnov test comparing current predicted probability distribution against training baseline	$p < 0.05$ → flag for review

PSI is chosen because it is interpretable (bounded, widely understood), works for both continuous and categorical features, and provides a single scalar per feature for dashboard monitoring.

In production these checks would be run via Evidently (open-source) or WhyLabs (SaaS) — both of which wrap PSI, KS, and a wider battery of drift and data-quality checks into a reporting layer with per-feature dashboards and alerting. The AWS-managed equivalent is SageMaker Model Monitor. The metadata.json written by arrears-train provides the per-feature reference distributions these tools would compare against, so no additional baseline-collection step is needed.

Concept drift monitoring

Check	Method	Threshold
Arrears base rate shift	Compare arrears_flag rate in new data against training baseline (~25%)	> 5 percentage point shift → trigger retraining review
Model performance on labelled new data	ROC-AUC and PR-AUC computed on ground-truth labels once available (~3 months lag)	> 0.05 absolute drop from training held-out value → review
Caseworker override rate	Proportion of households the team contacts that the model did not prioritise, or skips that the model did. Logged from the case-management system; no labels required.	> 30% override rate sustained for two consecutive weeks → user-research review and possible recalibration

The 3-month lag reflects the time between scoring and observing whether a household actually fell into arrears. This lag means monitoring is retrospective — a known limitation of any model predicting future events.

Fairness monitoring

Recompute sliced metrics (selection rate, TPR by disability, household type, age band) at each production run and compare against training baseline. Significant divergence — for example, TPR for disabled households falling more than 5 percentage points below baseline — should be reviewed before distributing the priority list.

Retraining triggers

- Arrears base rate shifts by more than 5 percentage points
- Feature PSI exceeds 0.25 for two or more features simultaneously
- ROC-AUC on labelled new data drops more than 0.05 from training baseline

- Major policy changes (Universal Credit rollout, housing benefit rules) that alter the feature distribution structurally

Tooling assumptions for production monitoring

In the current implementation, all monitoring checks require a data analyst to run manually. For a production system at scale, these would be automated via a scheduled job (e.g. GitHub Actions cron, Airflow DAG) writing results to a monitoring dashboard (e.g. Grafana, a Power BI report). The metadata.json artefact written per run provides the baseline values needed for drift comparison.